UNIVERSITY OF THE PEOPLE

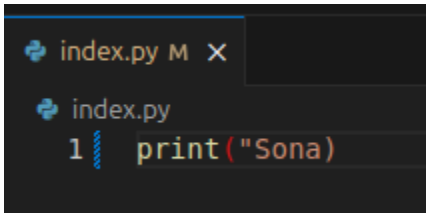
CS 1101 Unit 1

Programming Assignment

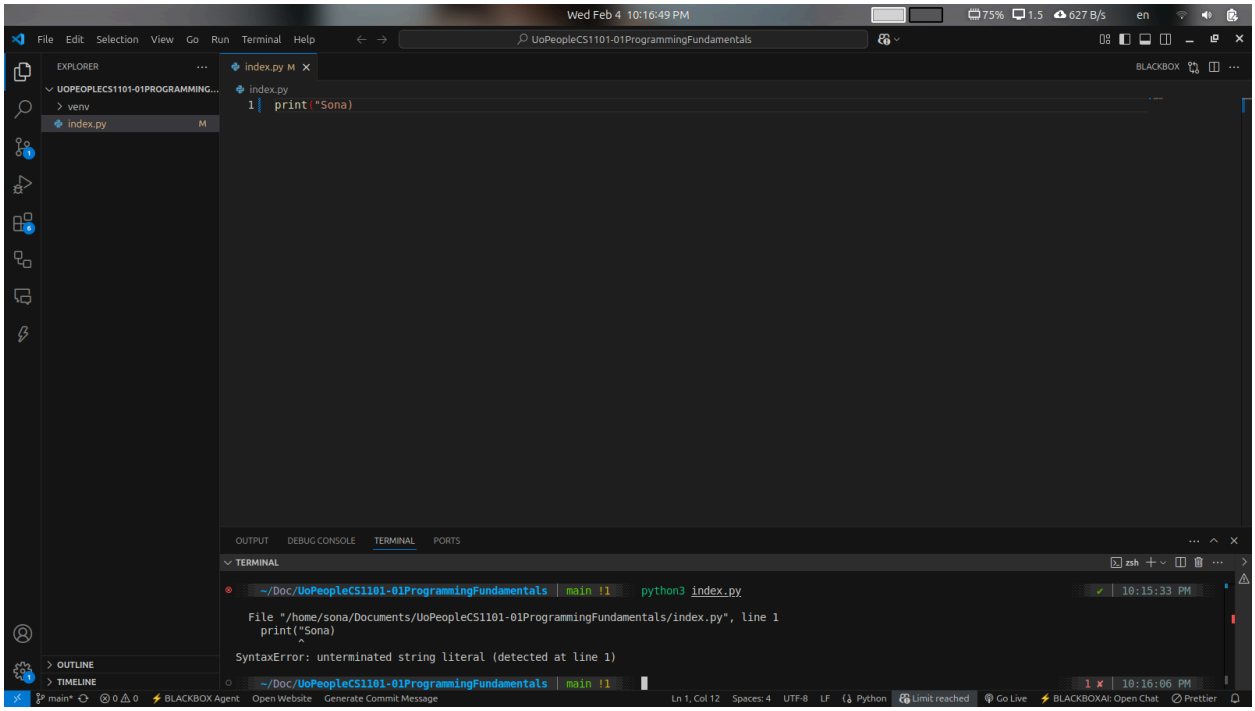
Assignment Response

1. **Answer the following questions (a) to (d).by explaining some frequently occurring errors. To help you answer the questions, write the code in Python and run it to produce output for each of the questions.**

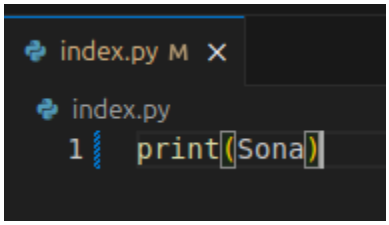
   a. If you are trying to print your name, what happens if you leave out one of the quotation marks or both, and why?



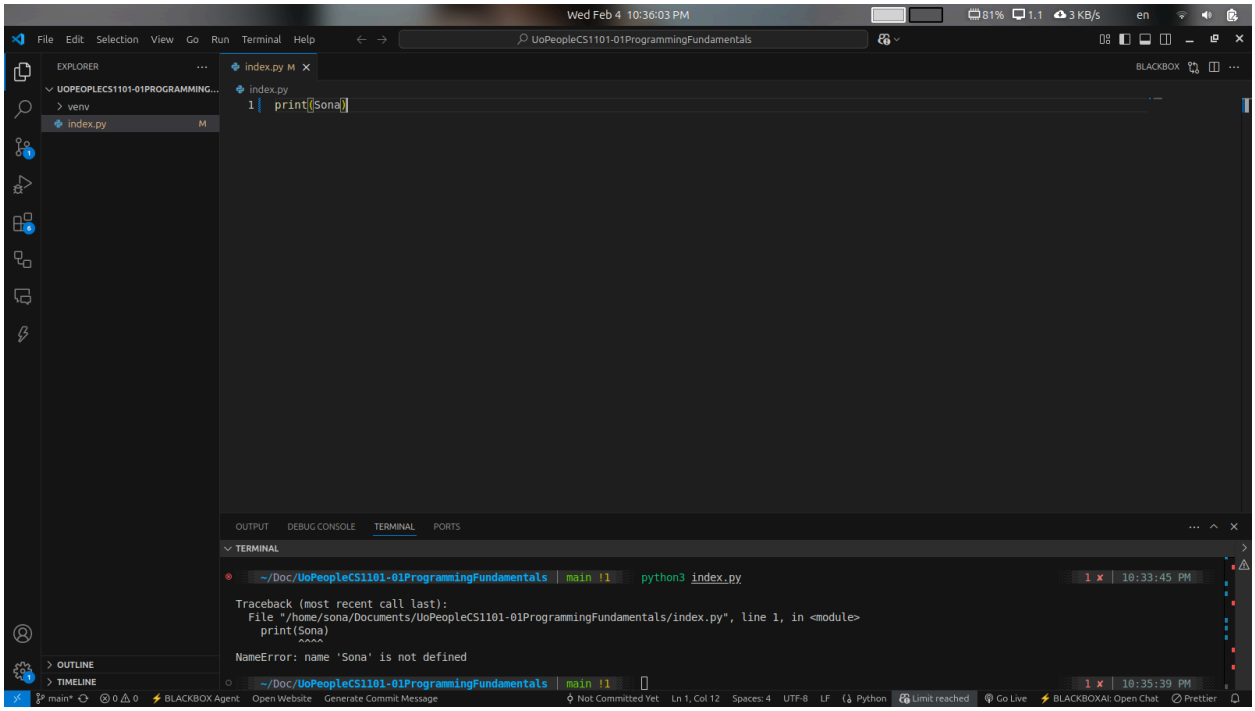
   This will give: SyntaxError: unterminated string literal



Why: In Python, strings must start and end with the same quotation marks (" or '). Here, the opening quote " exists but the closing quote is missing, so Python cannot determine where the string ends. As written in Downey (2015) "*The quotation marks in the program mark the beginning and end of the text to be displayed; they don't appear in the result*".



This will give: NameError: name 'Sona' is not defined



Why:

In Python, anything written without quotes is interpreted as a variable name.

Here, **Sona** is treated as a variable, but since we have never defined a variable called Sona in our program, Python cannot find it in memory.

Therefore, it raises a NameError, which is Python's way of saying: *"I don't know what Sona is."*.

b. What is the difference between * and ** operators in Python? Explain with the help of an example.

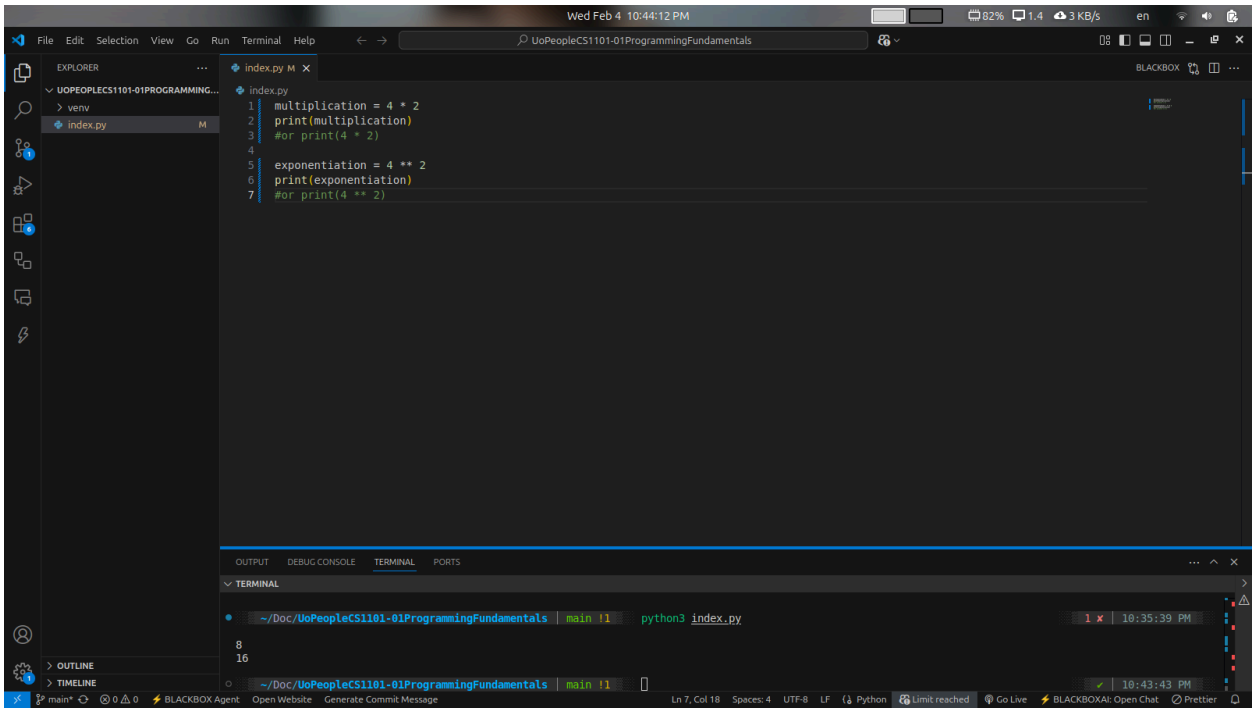
```python
multiplication = 4 * 2
print(multiplication)
#or print(4 * 2)

exponentiation = 4 ** 2
print(exponentiation)
#or print(4 ** 2)
```

This will give:

8

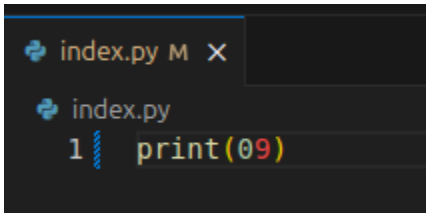
10



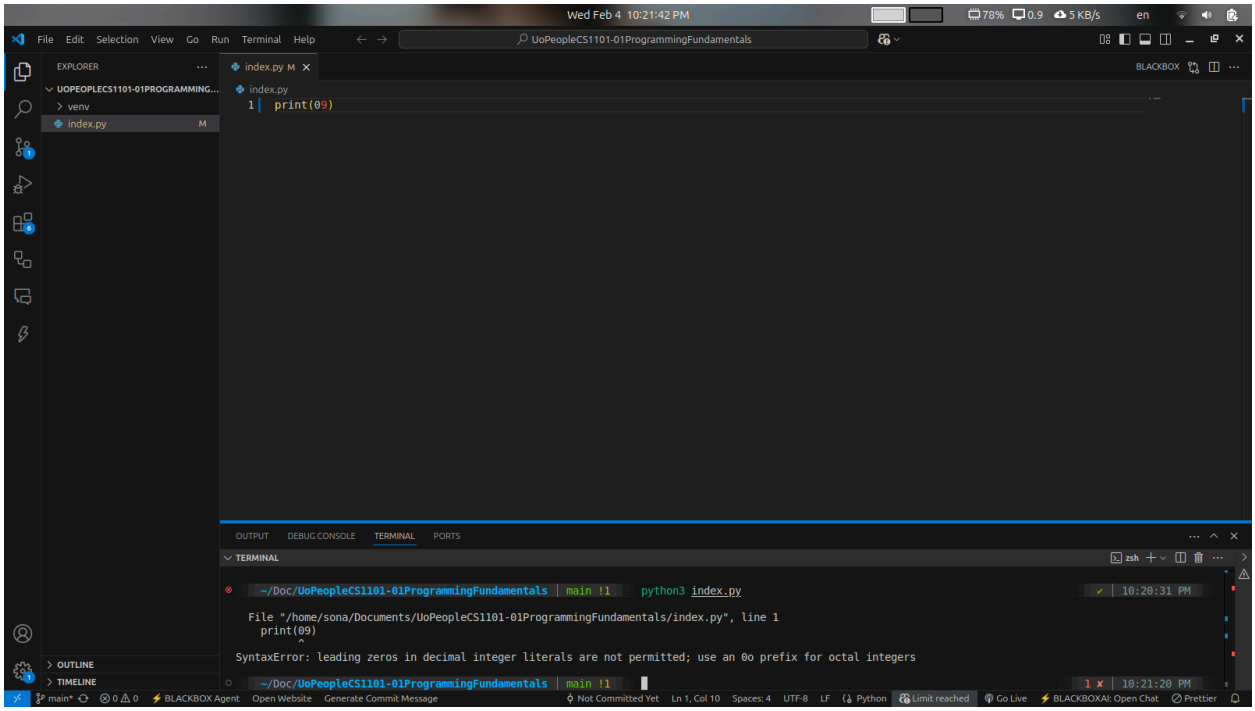
Why: In Python * is multiply and ** raises a number to a power, examples 4*2 is 8, 4**2 is 16. As written in Downey (2015) *"The operators +, -, and * perform addition, subtraction, and multiplication", "The operator ** performs exponentiation; that is, it raises a number to a power"*

c. In Python, is it possible to display an integer like 09? Justify your answer.

No, it's not possible to write an integer like 09 in Python because numbers cannot start with a leading zero.

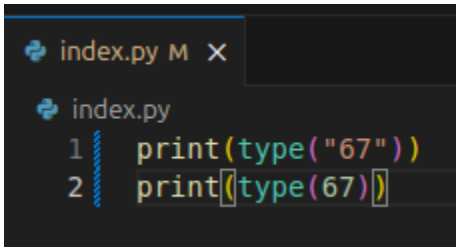


This will give: SyntaxError: leading zeros in decimal integer literals are not permitted; use an 0o prefix for octal integers



Why: Python interprets numbers without quotes as numeric literals, and a leading zero is not allowed in decimal integers because it could be confused with octal (base-8) notation in older Python versions.

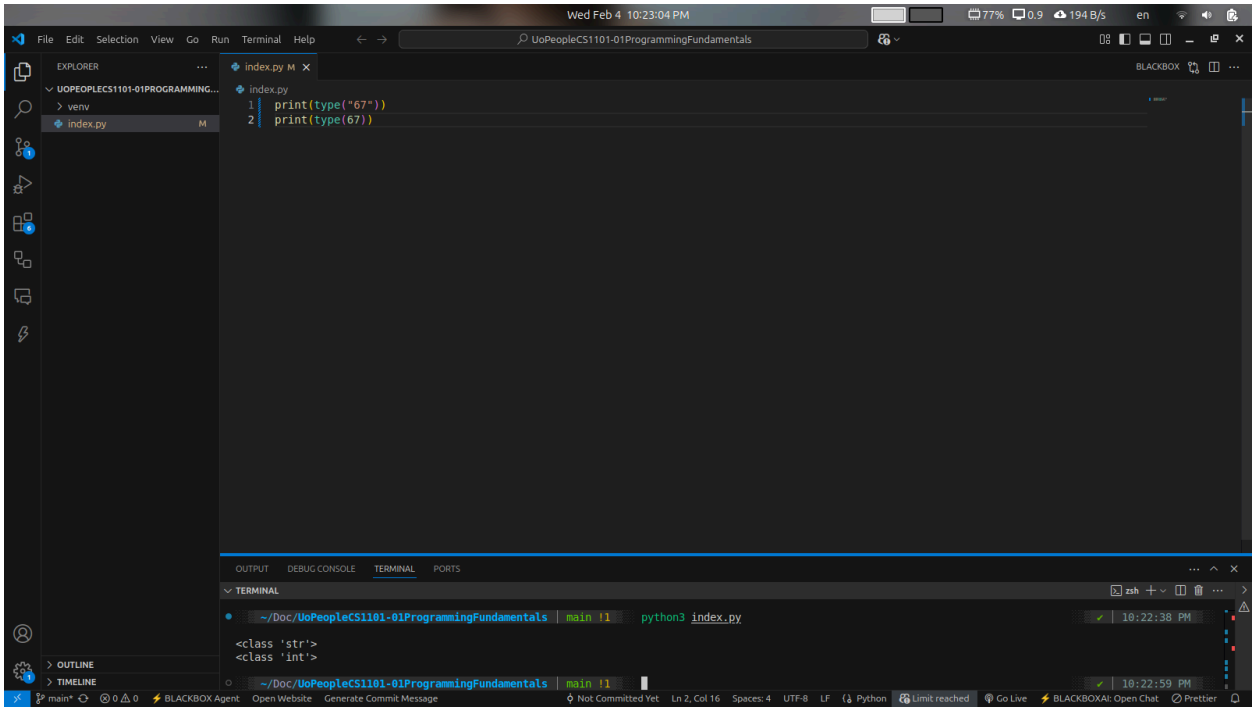
d. Run the commands type('67') and type(67). What is the difference in the output and why?



This will give:

<class 'str'>

<class 'int'>

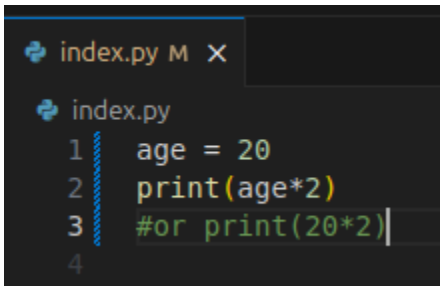


Why: As written in Downey (2015) *"Values belong to different types: 2 is an integer, 42.0 is a floating-point number, and 'Hello, World!' is a string, so-called because the letters it contains are strung together.", "In these results, the word "class" is used in the sense of a category; a type is a category of values".*

- '67' is a string because it's enclosed in quotes (' ' or " ").
- 67 is an integer because it's a numeric literal without quotes.

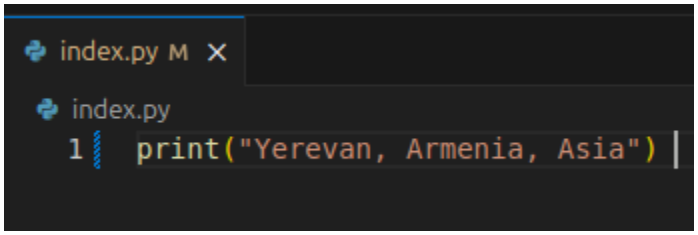
2. **Write a Python program for each of the following questions a to d.**

a. To multiply your age by 2 and display it. For example, if your age is 16, so 16 * 2 = 32



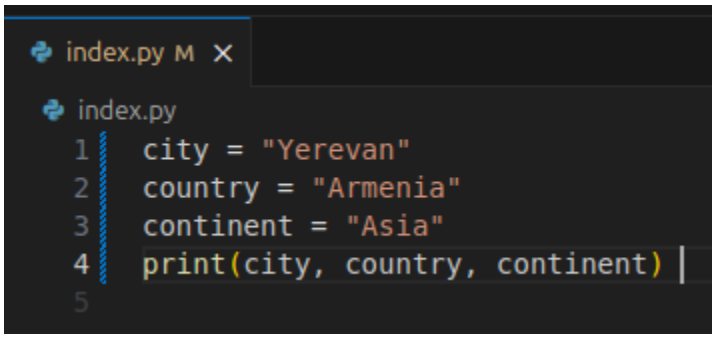
In this exercise, I learned how Python handles arithmetic operations using variables. By assigning my age (20) to a variable name age, I was able to perform a multiplication operation. This demonstrates that Python can store numerical data as integers and manipulate them using standard mathematical operators like the asterisk (*).

b. Display the name of the city, country, and continent you are living in.



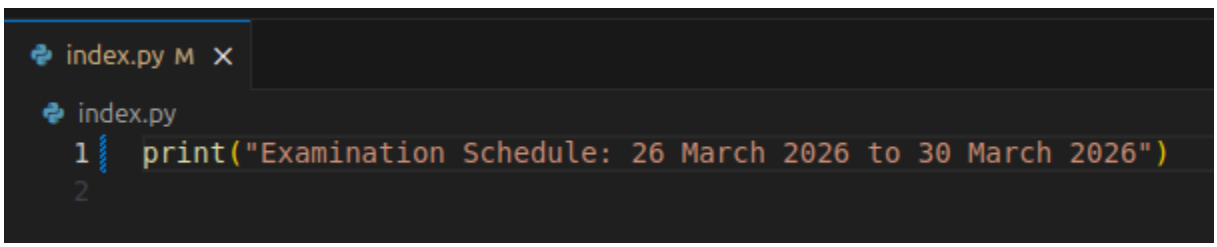
This experiment helped me understand the print() function's ability to output string literals. By enclosing my city, country, and continent in quotation marks, I instructed the

Python interpreter to treat the text as a single string data type. This is the most basic form of data output in Python.

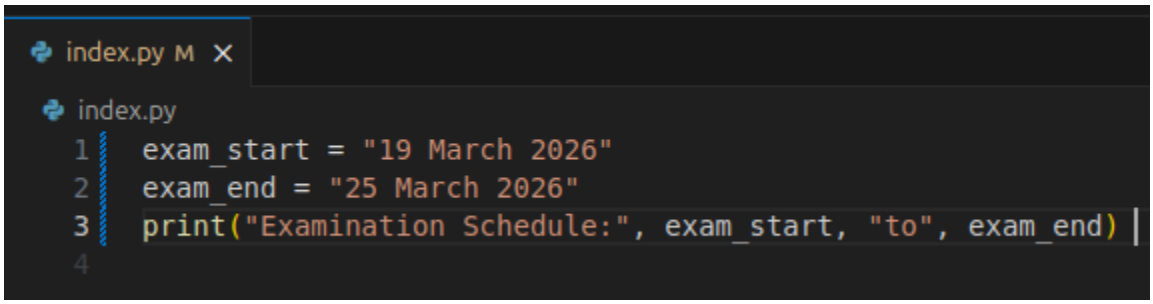


In this version of the program, I learned how to use variable assignment. Instead of printing a raw string, I assigned the values to three specific variables: city, country, and continent. When using the print() function with commas separating the variables, Python automatically adds a space between them in the output. This approach is much more professional because it allows the data (the names of the places) to be reused elsewhere in a larger program without retyping the text. It demonstrates an understanding of how Python allocates memory to store string data under specific identifiers.

c. To display the examination schedule (i.e., the starting and the ending day) of this term.



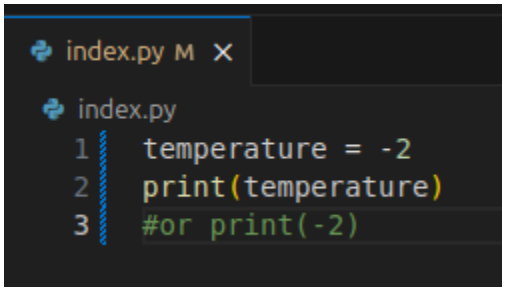
Creating the examination schedule output taught me that strings can contain a mix of alphabetic characters and numbers. Even though the dates contain numbers (e.g., 26, 2026), they are treated as text by Python because they are inside the quotes. This is useful for displaying formatted information to a user.



In this program, I utilized multiple variables to store specific date information as strings. By separating the start and end dates into the variables exam_start and exam_end, the code becomes more modular and easier to update in future terms.

When the print() function is called, it combines a fixed string literal ("Examination Schedule:") with the values stored in the variables and the connecting word "to". This demonstrates Python's ability to concatenate or join different pieces of information into a single line of output. I learned that using descriptive variable names makes the code more readable for other programmers, which is a key practice in professional software development.

d. Display the temperature of your country on the day the assignment is attempted by you



By displaying the current temperature, I practiced using Python to represent real-world, changing data. I learned that numerical values representing physical measurements can be easily printed to the console. In a more advanced program, this value could be stored as a "float" if it included decimal points, which is a common data type for scientific measurements like temperature.

References

Downey, A. (2015). Think Python: How to think like a computer scientist. Green Tree Press. https://greenteapress.com/thinkpython2/thinkpython2.pdf

Savage, B. (n.d.). 5. Using Python on a Mac. Python Documentation https://docs.python.org/3/using/mac.html

W3Schools. (n.d.). Python Variables. https://www.w3schools.com/python/python_variables.asp